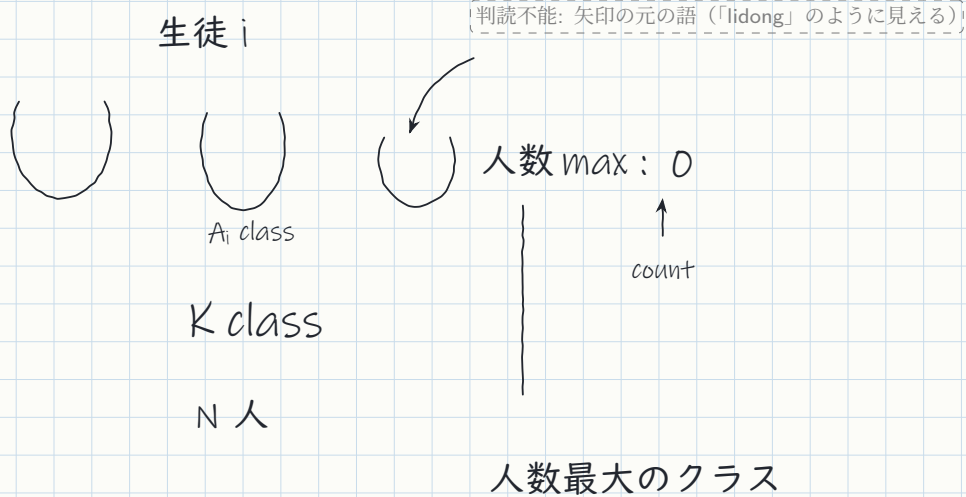




+1

入ったクラスは $c_i + 1$ 、ほかは最大 M のまま
 → 喜ぶ条件は $c_i + 1 \geq M$ 、つまり $c_i \geq M - 1$

class num sort

top next ..

AC 版の $\text{list}(\text{set}(a))$ は並びが保証されない
 人数 0, 1, 1 で 2 を出す (正しくは 1)

top, top - 1 の個数

結論は合っている。top - 1 は「値 $\text{max} - 1$ 」のこと
 → コードでは $b[-2]$ (2 番目に大きい人数) と読んで WA

abc473 C — Change Schools

- 問題: https://atcoder.jp/contests/abc473/tasks/abc473_c
- 提出 (2026-08-29, Python) : WA → AC
- 手書き原本: Box/Photo Backup/手書き/atcoder/abc473/c/ (20260925_174830.jpg の 1 枚。撮影 2026-09-25。前の 1 ページが再現。赤は後から入れた振り返り)

メモで考えたこと

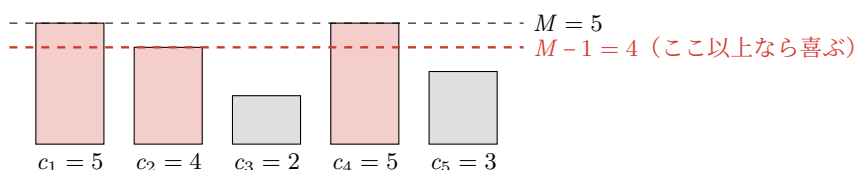
メモでは、クラスを袋として描いて「K class / N 人」「人数 max のクラス」「count」を置き、高橋くんが入るとそのクラスが +1 になることを書いた。そのうえで「class num sort → top, next ...」と並べ、「top, top -1 の個数」と結論を書いた。この結論 (人数が max か max -1 のクラスの数) は正しい。

つまずき

- WA:** コードでは「top -1」を「2 番目に大きい人数」として数えていた (`b[-2]`)。2 番目に大きい人数が max -1 より小さいとき、そのクラスに入っても max のクラスに負けるので数えてはいけない。メモの「top -1」は値としての max -1 なのに、コードで「順位としての 2 番目」に読み替えてしまった。
- AC 版にも潜在的なバグがある:** `b = list(set(a))` はソートされる保証がない。小さい整数ではたまたま昇順に並ぶが、`list(set([1, 8]))` は `[8, 1]` になる。実際に 10 3 / 1 1 1 1 1 1 1 1 2 3 (人数 8, 1, 1) を AC 版に入れると 2 を出す (正しくは 1)。テストケースにこの形が無かったので通っただけ。

正しい考え方

各クラスの人数を c_i 、 $M = \max c_i$ とする。クラス i に入ると $c_i + 1$ になり、ほかのクラスは最大 M のまま。喜ぶ条件は $c_i + 1 \geq M$ 、つまり $c_i \geq M - 1$ 。これを満たすクラスを数えるだけでよい (ソートも set も要らない)。たとえば人数 5, 4, 2, 5, 3 ($M = 5$) なら、赤の点線 ($M - 1$) 以上のクラスを数える。



```
n, k = map(int, input().split())
c = [0] * k
for x in map(int, input().split()):
    c[x - 1] += 1
m = max(c)
print(sum(1 for v in c if v >= m - 1))
```

次に活かすこと

- メモの結論を条件式 ($c_i \geq M - 1$) の形で書いてからコードにする。「top -1 の個数」のような言葉のままだと、コードで別の意味に化ける。
- 並び順が欲しいときは `sorted(set(a))`。set の順序に頼らない。